

scraps Pipeline Change Report — since 94e9782

Branch `replace-umitools-pysam`. Baseline commit `94e9782` ("Add de novo RT priming site discovery module", 2026-06-16). This report summarizes every change on the branch through commit `4e44ec4` (2026-07-11), combining the commit record with the design rationale developed during the work.

Executive summary

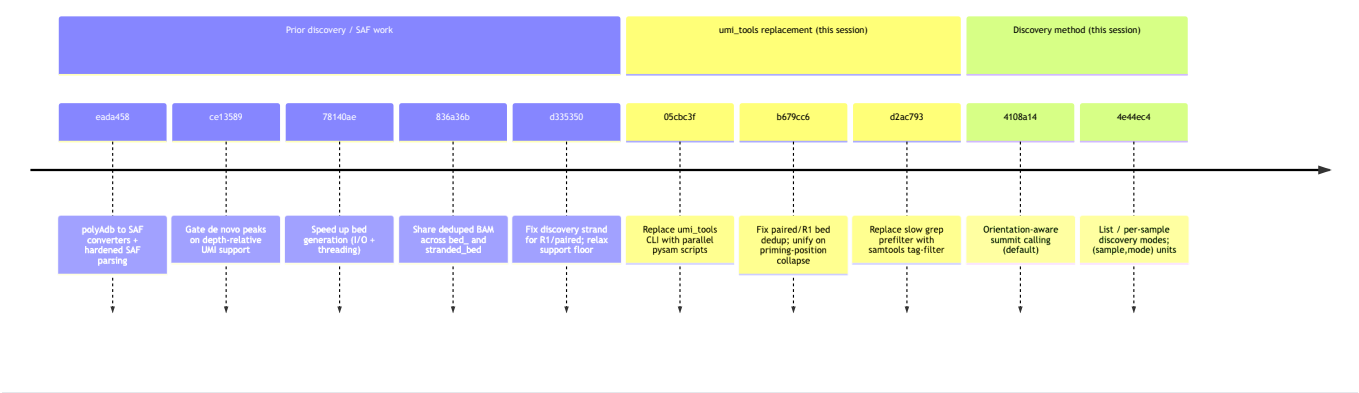
Ten commits since the baseline reshaped the counting and discovery portions of the pipeline. The headline outcomes:

- Both single-threaded `umi_tools` bottlenecks removed and replaced with parallel `pysam` scripts. The count table is reproduced **bit-for-bit**; the bed/discovery pileups adopt a corrected, molecule-level deduplication.
- Large speedups on a real 28M-read library: UMI counting **92 s** → **24 s** (8 threads), bed generation **604 s** → **72 s**, and the barcode prefilter **311 s** → **54 s**.
- A corrected paired/R1 deduplication semantic — collapse PCR duplicates by `(cell, UMI, priming position)` rather than by fragment span — that also makes the bed pileups consistent with the count table.
- A more accurate discovery summit caller that snaps the called site onto the cleavage boundary; exact agreement with `polyAdb` rose from **~39% to ~76%**.
- Flexible discovery modes: a single mode, a list of modes, per-sample overrides, and arbitrary `(sample, mode)` pooling units.
- Supporting work: `polyAdb`→SAF converters, hardened SAF parsing, depth-relative peak gating, and a synthetic + real-data validation harness.

⚠ **Correctness posture.** Count tables are bit-identical to the previous pipeline. Bed and discovery outputs change by design (molecule-level dedup and a new summit rule); both changes were validated and are documented below.

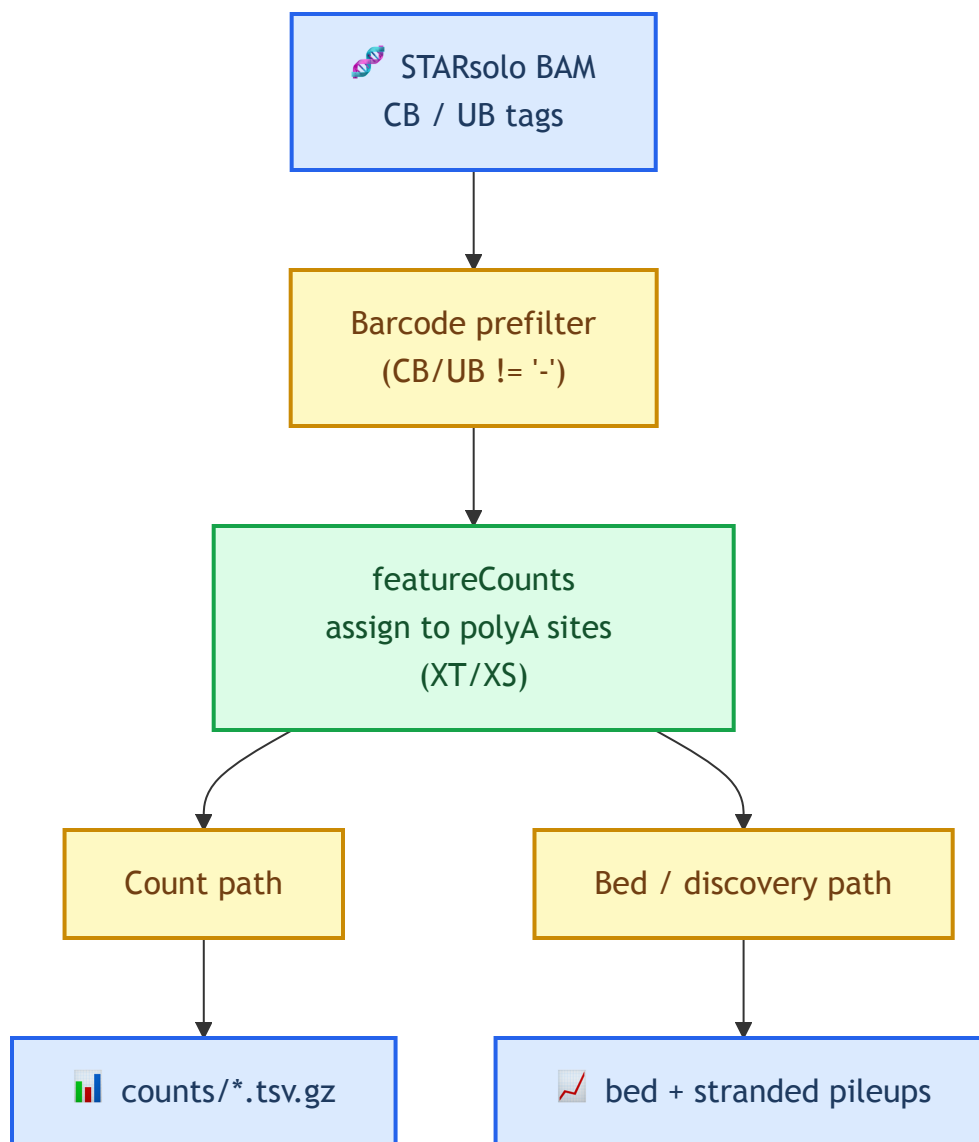
Commit timeline

Chronological list of the ten commits between the baseline and the branch head. The final four (bottom of the diagram) were produced during the working session that this report accompanies.



The counting pipeline before and after

The `assign` → `count` → `bed` data flow. `featureCounts` is retained; the two `umi_tools` invocations and the slow prefilter are replaced.



Step	Before (94e9782)	After (branch head)	Result
Barcode prefilter	<code>samtools view -h \ grep -v 'CB:Z:-\ UB:Z:-' \ samtools view -b</code>	<code>samtools view -e '[CB]!="-' && [UB]!="-'</code>	311 s → 54 s (5.8×), read set bit-identical
Site assignment	<code>featureCounts</code>	<code>featureCounts</code> (unchanged)	~25 s; retained deliberately
UMI count	<code>umi_tools count</code> (single-thread)	<code>count_sites.py</code> (pysam, per-contig parallel)	92 s → 24 s, bit-identical
Dedup + pileup	<code>umi_tools dedup</code> + <code>bedtools genomecov</code>	<code>pileup_sites.py</code> (single pass)	604 s → 72 s, corrected semantics



Replacing umi_tools with pysam

Commits `05cbc3f` , `b679cc6` , `d2ac793` .

Why

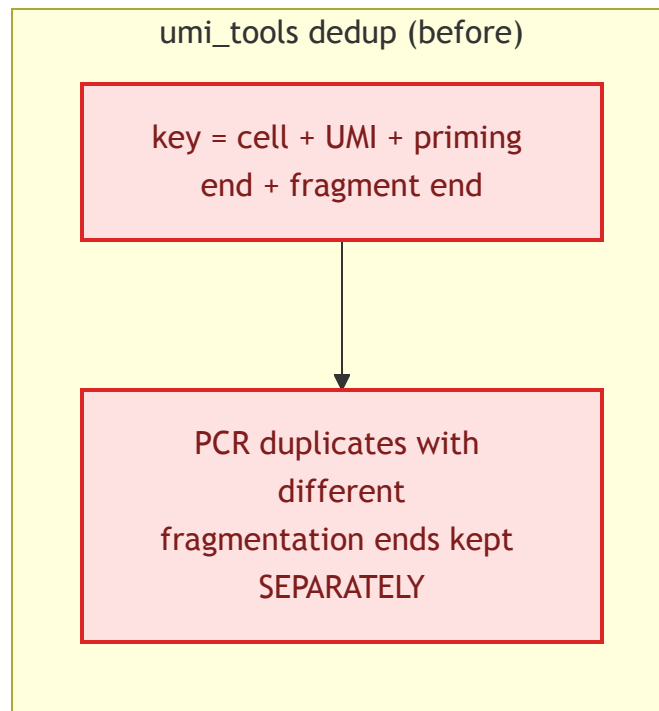
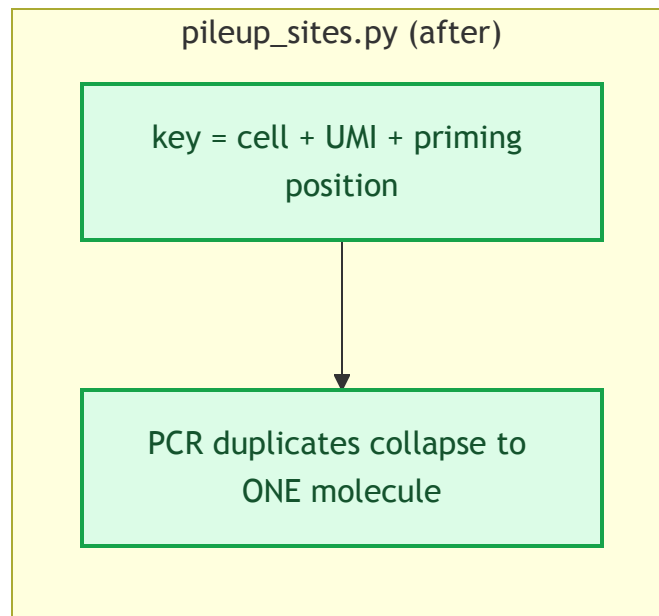
Both `umi_tools count` and `umi_tools dedup` are single-threaded and were the dominant runtime cost. `featureCounts` , by contrast, is only ~25 s and mature, so it was kept; a bit-exact reimplementation was judged not worth the risk.

`count_sites.py` — bit-exact directional counting

`umi_tools count` runs the **directional** UMI-network method by default (not plain unique counting). To reproduce it exactly, `count_sites.py` groups reads by `(cell, featureCounts XT site)` and calls **umi_tools' own UMIClusterer** per group, parallelized across contigs. Reusing the library guarantees the directional collapse matches. On real data the output was bit-identical (1,721,298 rows; sum 26,098,359).

`pileup_sites.py` — corrected molecule dedup

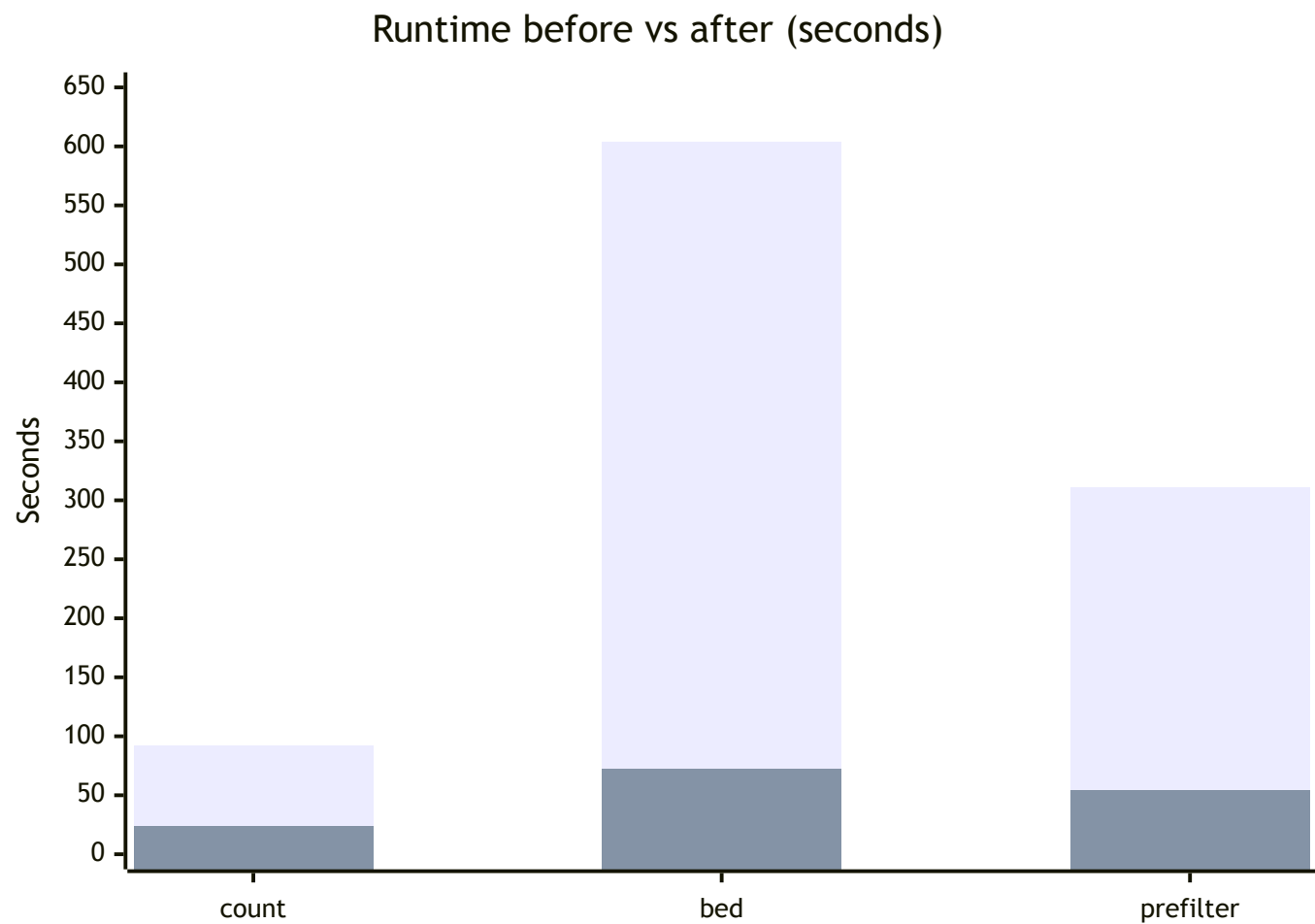
The bed path replaces `umi_tools dedup --method=unique` + `genomecov` . The key semantic question surfaced on real paired data:



The opposite (non-priming) end of a molecule is set by **fragmentation after PCR amplification**, so reads sharing a cell, UMI, and priming site but differing at that end are PCR duplicates of one molecule. `umi_tools` re-inflates them; `pileup_sites.py` deduplicates on (cell, UMI, priming position) uniformly across R1/R2/paired, which is both correct for the chemistry and consistent with the count table. Non-proper-pair reads are discarded to match the legacy read set.

This reduced the discovery stranded-bed signal by ~37% on the real AEG1 library — larger than the ~3.7% seen on sparse synthetic data because real libraries have a much higher PCR-duplicate multiplicity. The effect is the intended correction, not a regression.

Benchmark



Path	Before (s)	After (s)	Speedup
UMI count	92	24 (8 threads)	3.8×
Bed / dedup pileup	604	72	8.4×
Barcode prefilter	311	54	5.8×

Orientation-aware summit calling

Commit `4108a14` .

The problem

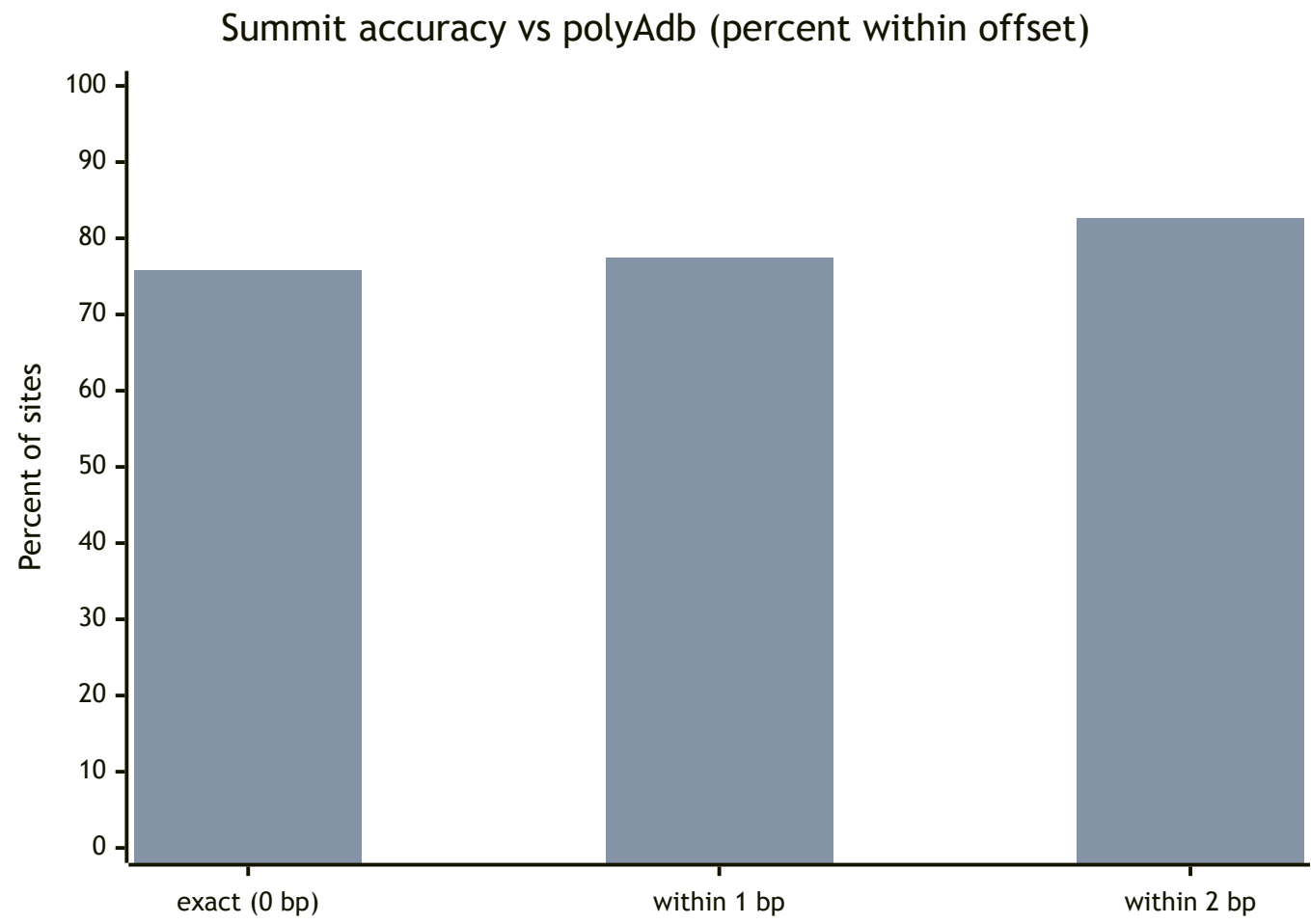
3'-end read pileups have a **hard boundary at the cleavage site** and **tail upstream** (transcript orientation). A Gaussian KDE places the summit at the smoothed centre of mass — ~1–2 bp upstream of the true cleavage site — and matched the polyAdb position exactly only ~39% of the time.

The fix

KDE is retained for **cluster detection and UMI support** (the called site set is unchanged), but the **summit within each cluster** is now placed by a new rule: the most-downstream (transcript-3') base whose depth is at least `summit_frac` × cluster peak

depth, searched over the full occupied read pileup. This snaps the summit onto the cleavage boundary. `summit_frac = 0.75` was chosen by sweeping 0.3–1.0 against polyAdb. Both methods remain available via `DISCOVERY.summit_method` (`downstream` default, `kde` legacy).

Summit accuracy vs polyAdb cleavage points (`known_pas`), same 277,929 sites called by both methods. Higher exact-match is better.

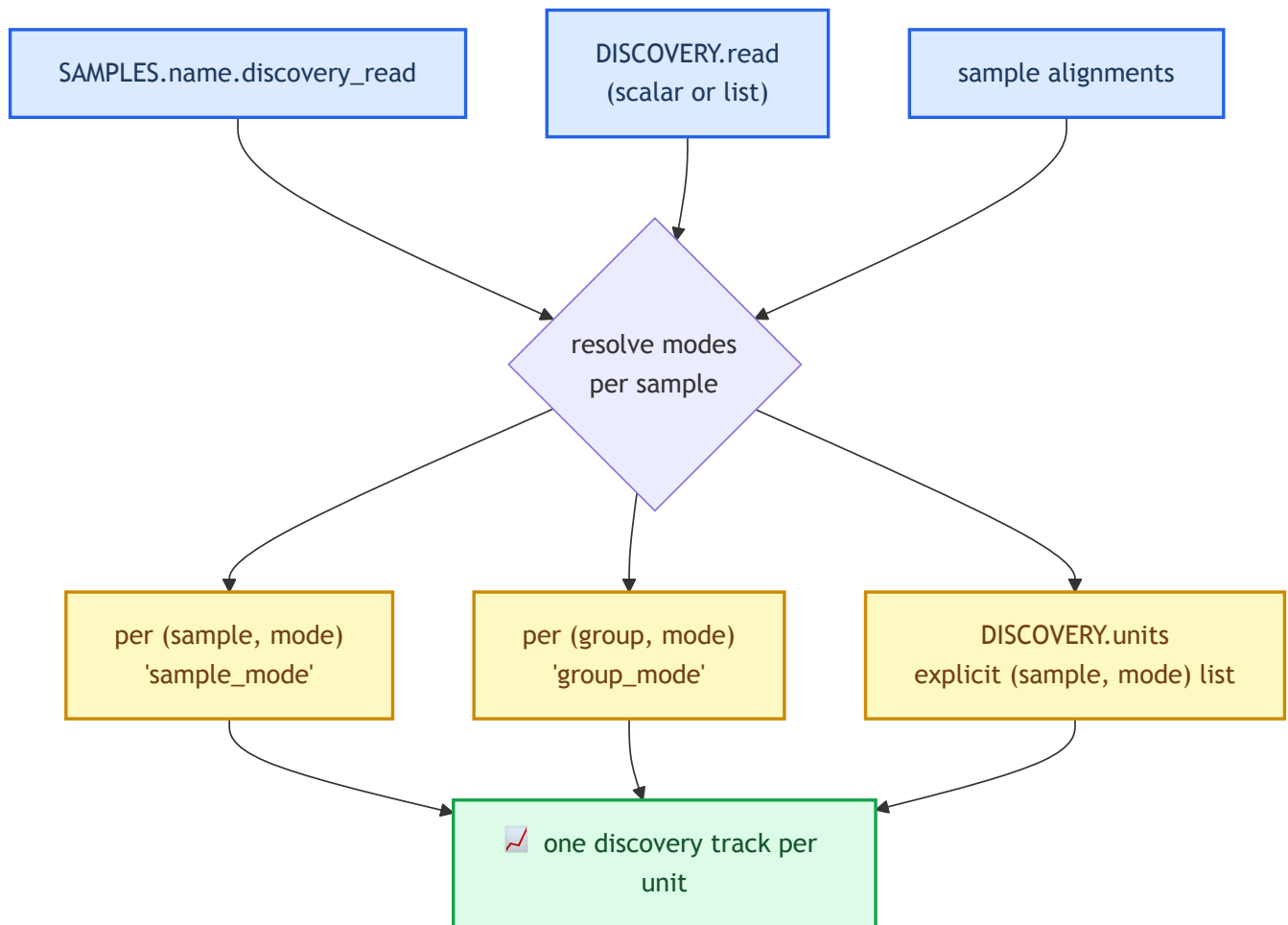


Method	Exact (0 bp)	≤1 bp	≤2 bp	Mean offset
kde (legacy)	39.2%	61.9%	77.9%	−1.12 bp
downstream (0.75, default)	75.8%	77.5%	82.7%	−0.43 bp

Flexible discovery modes

Commit `4e44ec4` .

The single scalar `DISCOVERY.read` became a flexible scheme. Modes per sample resolve as: per-sample `discovery_read` override → global `read` (scalar or list) → the sample's own `alignments` , then intersected with what the sample was actually aligned in.



- **Per (sample, mode)** — default track `<sample>_<mode>` .
- **Per (group, mode)** — `DISCOVERY.groups` pools member samples within each shared mode → `<group>_<mode>` .
- **Explicit DISCOVERY.units** — aggregate an arbitrary list of `(sample, mode)` beds, across samples and/or modes.

Because stranded beds share a common transcript-oriented coordinate system, any set can be summed. **Caveat:** pooling multiple modes derived from the same underlying reads double-counts UMI support; only pool independent measurements. Existing scalar- `read` and `groups` configs keep working; the only behavioral change is that per-sample tracks are now mode-suffixed.

Validation

How each output class is guarded. Count tables must be bit-exact; beds follow a directional criterion since their semantics intentionally changed.

Output	Criterion	Status
<code>counts/*.tsv.gz</code>	Bit-identical to <code>umi_tools count</code>	✅ verified (real + synthetic)
<code>bed/*</code> and stranded beds	Directional: no new positions, <code>new_sum ≤ old_sum</code>	✅ verified (real + synthetic)
Discovery summit	Offset vs polyAdb tightened, site set unchanged	✅ verified on AEG1
Snakemake DAG	<code>snakemake -npr</code> clean across mode scenarios	✅ 0 errors

Supporting harness added this branch: `tests/generate_synthetic.py` (synthetic assigned BAMs with soft-clips, splices, PCR-duplicate and non-proper-pair edge cases), `tests/validate_synthetic.sh`, and `tests/validate_realdata.sh` (old-vs-new diff on a real run).

Files changed

Aggregate diffstat across all ten commits: 22 files, +2,307 / -239 lines.

Area	Files
New pysam scripts	<code>inst/scripts/count_sites.py</code> , <code>inst/scripts/pileup_sites.py</code>
Discovery caller	<code>inst/scripts/kde_peaks.py</code> , <code>inst/scripts/annotate_sites.py</code>
polyAdb converters	<code>inst/scripts/polyadb/{pas32_to_saf,pasv4_to_saf,saf_common,validate_saf}.py</code> , <code>polyadb/README.md</code>
Workflow rules	<code>Snakefile</code> , <code>rules/count.snake</code> , <code>rules/discovery.snake</code> , <code>rules/check_versions.snake</code>
Config / docs	<code>config.yaml</code> , <code>docs/discovery.md</code> , <code>AGENTS.md</code> , <code>README.md</code>
Tests	<code>tests/generate_synthetic.py</code> , <code>tests/validate_synthetic.sh</code> , <code>tests/validate_realdata.sh</code>
Environment	<code>scraps_conda.yml</code>
R helper	<code>inst/scripts/R/scraps_to_seurat.R</code>

Net assessment

The branch removes the pipeline's real runtime bottlenecks while keeping the primary quantitative output (the count matrix) bit-identical. The bed and discovery changes are deliberate scientific corrections — molecule-level deduplication and cleavage-boundary summit placement — each validated against real data and polyAdb, and each documented with its caveats. Discovery configuration is now substantially more flexible without breaking existing configs.